LLMs for Social Science at Scale

TechUP at GU — 6th March 2026

Ashrakat Elshehawy
Assistant Professor, University College London

LLMs for Social Science at Scale

We deal with messy data all the time: 1. scanned archives, 2. piles of text that need coding, 3. locations buried in paragraphs.

LLMs can help with a lot of this, but the question is always: *did the model actually get it right?*

This notebook covers four things, and every one includes a check against ground truth or a traditional tool (there are many ways to validate):

1. **LLM basics** — prompt engineering, prompt sensitivity
2. **Text classification** — zero-shot LLM, model comparison, validation
3. **Named entity recognition** — spaCy vs LLM
4. **Vision OCR** — GPT vs Gemini on a historical document

```
## install what we need (run manually — not during render)
!pip install openai google-generativeai pandas scikit-learn matplotlib seaborn folium spacy -q
## medium model has 300-dim word vectors; small model does NOT
!python -m spacy download en_core_web_md -q
```

API Keys

We'll use two AI services: OpenAI (for GPT) and Google (for Gemini). Both need an API key — think of it as a password that lets your code talk to their servers.

Get an OpenAI key at platform.openai.com/api-keys. Get a Google key at aistudio.google.com/apikey — this one's free. Paste them below, and never share a notebook with your real key in it.

```python
from openai import OpenAI
import google.generativeai as genai
import json

## paste your keys here
OPENAI_KEY = "YOUR-OPENAI-KEY-HERE"
GEMINI_KEY = "YOUR-GEMINI-KEY-HERE"

## create API client objects — we'll reuse these throughout
client = OpenAI(api_key=OPENAI_KEY)
genai.configure(api_key=GEMINI_KEY)

   # Send a system prompt + user text to a GPT model and return the string response.

   # Parameters
   # ----------
   # prompt      : str   — system-role instructions (your annotation schema / task definition)
   # text        : str   — user-role content (the document/text to be processed)
   # model       : str   — OpenAI model identifier
   # temperature : float — sampling temperature; lower = more deterministic output.


def ask_gpt(prompt, text, model="gpt-4o-mini", temperature=0.1):
    """Send a system prompt + user text to GPT and return the response."""
    kwargs = {
        "model": model,
        ## system = your instructions, user = the data to process
        "messages": [
            {"role": "system", "content": prompt},
            {"role": "user", "content": text}
        ],
    }
    ## some models only support default temperature
    if temperature is not None:
        kwargs["temperature"] = temperature
    try:
        response = client.chat.completions.create(**kwargs) #with or without dic
    except Exception:
        ## retry without temperature if the model doesn't support it
        kwargs.pop("temperature", None)
        response = client.chat.completions.create(**kwargs)
    return response.choices[0].message.content
```

```python
def parse_json_response(raw):
    """LLMs sometimes wrap JSON in code blocks. Strip that and parse."""
    # Remove leading/trailing whitespace from the raw model output.
    # This handles newlines or spaces that often appear before/after code blocks.
    clean = raw.strip()
    if clean.startswith("```"):
        ## strip the ```json ... ``` wrapper that GPT likes to add
            # GPT often returns JSON wrapped in a markdown code fence, e.g.:
        #   ```json
        #   {"label": "positive", "confidence": 0.9}
        #   ```
        # We need to strip that wrapper before parsing.

        # Split on the first newline only, discarding the opening fence line
        # (e.g. "```json" or "```").
        # [1] takes everything AFTER that first line.
        clean = clean.split("\n", 1)[1]
        # Strip the closing ``` fence from the end.
        # rsplit from the right ensures we only remove the last occurrence,
        # in case the JSON content itself contains backticks.
        clean = clean.rsplit("```", 1)[0]
        #  # Parse the cleaned string into a Python dict/list.
    return json.loads(clean)


print("Ready.")


#The core logic handles this transformation:
#Input (raw model output):
#```json
#  {"label": "protest", "confidence": 0.87}
#```
#
#After .strip()          →  "```json\n{...}\n```"
#After .split("\n", 1)   →  '{"label": "protest", "confidence": 0.87}\n```'
#After .rsplit("```", 1) →  '{"label": "protest", "confidence": 0.87}\n'
#After json.loads()      →  {"label": "protest", "confidence": 0.87}
```

```
Ready.
```

Part 1: What Are LLMs and How Do We Use Them?

An LLM is basically a neural network trained on a huge amount of text. It learns to predict what comes next in a sequence, and it turns out that's useful for all sorts of tasks:

- Classification,
- Extraction,
- Translation,
- Summarization.

We talk to it through an API: send a prompt and some text, get a structured response back.

> ⚠️ **Important caveat**
>
> LLMs *generate* text. They can (and do) hallucinate: producing confident output that's just wrong. This is why we pair every application with a battery of verification steps.

Prompt engineering basics

When you call an LLM through the API, you send messages with roles:

- **System message** — your instructions ("classify this text into categories X, Y, Z…")
- **User message** — the actual data you want processed
- **Temperature** — controls how random or varied the model's output is. We keep it low (0.1) for consistency.
- **Structured output** — ask for JSON so you can parse the result in code

```python
## quick test — does the API work?

result = ask_gpt(
    prompt="Classify this political text as one of: economy, security, social_policy, environment. Return ONLY the label.",
    text="The parliament approved a binding target to reduce carbon emissions by 55% before 2030."
)

print(f"GPT says: {result}")
```

```
GPT says: environment
```

Prompt sensitivity — this matters more than you'd think

Trivial changes to how you word a prompt can flip the output. Capitalizing the options, adding bold formatting, telling the model what *not* to do can all make changes to the output.

If your classification changes because you bolded a word, that's a problem. Let's see how bad this actually is.

```python
pd.set_option('display.max_colwidth', None)

df.head()
```

	review
0	A masterpiece. The acting, direction, and score are all flawless.
1	Absolutely hilarious from start to finish. Best comedy of the year.
2	Unwatchable. I turned it off after thirty minutes of wooden dialogue.
3	A complete disaster — incoherent plot, terrible CGI, zero likable characters.
4	It was fine. Not great, not terrible. A perfectly average way to spend two hours.

```python
import time
import pandas as pd
from collections import Counter

## five ways to ask the same question
prompts = {
    "A_plain":      "Classify the sentiment of this movie review as positive, negative, or neutral. Return ONLY the label.",
    "B_CAPS":       "Classify the sentiment of this movie review as POSITIVE, NEGATIVE, or NEUTRAL. Return ONLY the label.",
    "C_bold":       "Classify the sentiment of this movie review as **positive**, **negative**, or **neutral**. Return ONLY the label.",
    "D_no_neutral": "Classify the sentiment of this movie review as positive or negative. Do NOT use neutral. Return ONLY the label.",
    "E_reversed":   "Classify the sentiment of this movie review as neutral, negative, or positive. Return ONLY the label.",
}

## run all combinations
results = {} ## stores labels for each prompt strategy: {prompt_name: [label, label, ...]}
print(f"Running {len(reviews)} reviews × {len(prompts)} prompts = {len(reviews)*len(prompts)} API calls...\n")

for name, prompt in prompts.items():        # iterate over each prompt strategy
    labels = []                             # collect labels for this prompt
    for review in reviews:                  # iterate over each review text
        label = (
            ask_gpt(prompt, review)
            .strip()                        # remove leading/trailing whitespace
            .lower()                        # normalize to lowercase
            .replace("*", "")              # remove stray asterisks (markdown bold artifacts)
        )
        labels.append(label)
        time.sleep(0.3)                     # avoid hitting API rate limits

    results[name] = labels                  # store this prompt's labels in results dict
    print(f"  {name} done")

print("\nAll done.")
```

```
Running 12 reviews × 5 prompts = 60 API calls...

  A_plain done
  B_CAPS done
  C_bold done
  D_no_neutral done
  E_reversed done

All done.
```

```python
## compare results across prompt variants
short = [r[:55] + "…" if len(r) > 55 else r for r in reviews]  ## truncate for display
df = pd.DataFrame(results, index=short)

## flag rows where not all 5 variants agree — set() removes duplicates
df["all_agree"] = df.apply(lambda row: "✓" if len(set(row)) == 1 else "✗", axis=1)

n_disagree = sum(1 for _, row in df.iterrows() if row["all_agree"] == "✗")
print(f"{n_disagree} out of {len(reviews)} reviews got different labels depending on prompt phrasing.\n")
df
```

```
4 out of 12 reviews got different labels depending on prompt phrasing.
```

	A_plain	B_CAPS	C_bold	D_no_neutral	E_reversed	all_agree
A masterpiece. The acting, direction, and score are all…	positive	positive	positive	positive	positive	✓
Absolutely hilarious from start to finish. Best comedy …	positive	positive	positive	positive	positive	✓
Unwatchable. I turned it off after thirty minutes of wo…	negative	negative	negative	negative	negative	✓
A complete disaster — incoherent plot, terrible CGI, ze…	negative	negative	negative	negative	negative	✓
It was fine. Not great, not terrible. A perfectly avera…	neutral	neutral	neutral	negative	neutral	✗
The first half is brilliant but the movie completely lo…	negative	negative	negative	negative	negative	✓
Visually gorgeous but emotionally empty. All style, no …	negative	negative	negative	negative	negative	✓
I can see why critics love it, but honestly it left me …	negative	negative	negative	negative	negative	✓
Not bad for a sequel. Doesn't reach the original but it…	neutral	neutral	neutral	positive	neutral	✗
My expectations were low and it mostly met them. A few …	neutral	neutral	neutral	negative	neutral	✗
An ambitious mess. I admire what the director tried eve…	neutral	neutral	neutral	negative	neutral	✗
The lead gives an Oscar-worthy performance but everythi…	negative	negative	negative	negative	negative	✓

- Testing small prompt variations is a good way to assess prompt sensitivity or robustness.
- Comparing against a hand-labeled sample is important if you want to know whether the prompt is not just stable, but also accurate
- Keeping a hold-out set that you never use during prompt iteration is also good practice. Analogy to train/test logic in supervised ML.

Zero-shot, few-shot, fine-tuning

- **Zero-shot** — just instructions, no examples. Works surprisingly well for many tasks.
- **Few-shot** — include some worked examples in the prompt. Helps with ambiguous cases.
- **Fine-tuning** — train the model on your own labeled data. Best results, most effort.

Part 2: Text Classification

This is probably the most common use case for LLMs in political science: classification.

Using LLMs: describe the categories and let the LLM provides labels.

There are many LLMs and they have different capabilities.

We will have to verify and validate this output consistently!

This is of course a mini example

```python
## 20 hand-labeled political texts — our ground truth for validation
## some are deliberately ambiguous (e.g. defense budget cuts → economy, not security)

test = [
    {"text": "The central bank cut the benchmark interest rate by a quarter point to stimulate lending.", "human_label": "economy"},
    {"text": "The country's GDP contracted for two consecutive quarters, officially entering a recession.", "human_label": "economy"},
    {"text": "Economic sanctions against the regime have crippled the banking sector and frozen foreign assets.", "human_label": "economy"},
    {"text": "Defense budget cuts forced major military contractors to lay off thousands of workers.", "human_label": "economy"},
    {"text": "Carbon taxes drove several energy-intensive manufacturers to relocate their factories overseas.", "human_label": "economy"},
    {"text": "The armed forces launched airstrikes against militant positions near the northern border.", "human_label": "security"},
    {"text": "NATO allies agreed to station additional troops along the eastern flank as a deterrent.", "human_label": "security"},
    {"text": "An intelligence report warned that water scarcity could trigger armed conflict in the region.", "human_label": "security"},
    {"text": "Arms manufacturers invested in solar-powered surveillance drones to cut fuel costs.", "human_label": "security"},
    {"text": "State-backed hackers targeted hospital computer networks in a coordinated cyberattack.", "human_label": "security"},
    {"text": "The government expanded subsidized school lunch programs to reach a million more children.", "human_label": "social_policy"},
    {"text": "A new law requires all employers to provide twelve weeks of paid parental leave.", "human_label": "social_policy"},
    {"text": "Refugee camps in the border region lack clean drinking water and basic medical supplies.", "human_label": "social_policy"},
    {"text": "Military families reported severe housing shortages on bases, with some in condemned buildings.", "human_label": "social_policy"},
    {"text": "Public health officials declared the air pollution crisis a national emergency, closing schools.", "human_label": "social_policy"},
    {"text": "Greenhouse gas emissions from the transportation sector increased by 8 percent last year.", "human_label": "environment"},
    {"text": "Deforestation in the tropical highlands accelerated due to illegal mining operations.", "human_label": "environment"},
    {"text": "Flooding caused by climate change destroyed critical infrastructure, costing the economy billions.", "human_label": "environment"},
    {"text": "The fishing industry collapsed after toxic industrial waste poisoned the coastal ecosystem.", "human_label": "environment"},
    {"text": "A drought linked to rising temperatures wiped out 60 percent of the country's wheat harvest.", "human_label": "environment"},
]

print(f"{len(test)} labeled texts for validation (5 per category)")
```

```
20 labeled texts for validation (5 per category)
```

Zero-shot classification

- We write a prompt listing our four categories with example topics so the LLM knows what we mean
- We loop over all 20 test texts, send each to GPT, and collect the predictions

```python
## list each category with example topics so the LLM knows what we mean
zero_shot_prompt = """Classify this political text into exactly one category:
- economy (taxes, trade, growth, employment, fiscal policy)
- security (defense, military, terrorism, borders, intelligence)
- social_policy (healthcare, education, welfare, housing, inequality)
- environment (climate, energy, emissions, biodiversity, pollution)

Return ONLY the category name, nothing else."""

import time
llm_labels = []
for item in test:
    ## strip + lowercase so "Economy" and "economy " both match
    label = ask_gpt(zero_shot_prompt, item["text"]).strip().lower()
    llm_labels.append(label)
    time.sleep(0.3)

print("LLM zero-shot predictions on test set:")
for item, pred in zip(test, llm_labels):
    match = "✓" if pred == item["human_label"] else "✗"
    print(f"  {match} predicted={pred:<15s} true={item['human_label']:<15s} {item['text'][:70]}...")
```

```
LLM zero-shot predictions on test set:
  ✓ predicted=economy         true=economy         The central bank cut the benchmark interest rate by a quarter point to...
  ✓ predicted=economy         true=economy         The country's GDP contracted for two consecutive quarters, officially ...
  ✓ predicted=economy         true=economy         Economic sanctions against the regime have crippled the banking sector...
  ✗ predicted=security        true=economy         Defense budget cuts forced major military contractors to lay off thous...
  ✓ predicted=economy         true=economy         Carbon taxes drove several energy-intensive manufacturers to relocate ...
  ✓ predicted=security        true=security        The armed forces launched airstrikes against militant positions near t...
  ✓ predicted=security        true=security        NATO allies agreed to station additional troops along the eastern flan...
  ✓ predicted=security        true=security        An intelligence report warned that water scarcity could trigger armed ...
  ✗ predicted=environment     true=security        Arms manufacturers invested in solar-powered surveillance drones to cu...
  ✓ predicted=security        true=security        State-backed hackers targeted hospital computer networks in a coordina...
  ✓ predicted=social_policy   true=social_policy   The government expanded subsidized school lunch programs to reach a mi...
  ✓ predicted=social_policy   true=social_policy   A new law requires all employers to provide twelve weeks of paid paren...
  ✓ predicted=social_policy   true=social_policy   Refugee camps in the border region lack clean drinking water and basic...
  ✓ predicted=social_policy   true=social_policy   Military families reported severe housing shortages on bases, with som...
  ✗ predicted=environment     true=social_policy   Public health officials declared the air pollution crisis a national e...
  ✓ predicted=environment     true=environment     Greenhouse gas emissions from the transportation sector increased by 8...
  ✓ predicted=environment     true=environment     Deforestation in the tropical highlands accelerated due to illegal min...
  ✓ predicted=environment     true=environment     Flooding caused by climate change destroyed critical infrastructure, c...
  ✓ predicted=environment     true=environment     The fishing industry collapsed after toxic industrial waste poisoned t...
  ✓ predicted=environment     true=environment     A drought linked to rising temperatures wiped out 60 percent of the co...
```

Validation — how did it do?

Performance metrics

```python
## how did the LLM do?
## precision = of the items predicted as X, how many are really X?
## recall    = of the items that are really X, how many did we find?
## F1        = harmonic mean of precision and recall (one number summary)
from sklearn.metrics import classification_report, confusion_matrix, f1_score
import numpy as np

human_labels = [d["human_label"] for d in test]
categories = ["economy", "security", "social_policy", "environment"]

print(classification_report(human_labels, llm_labels, labels=categories))
```

```
              precision    recall  f1-score   support

     economy       1.00      0.80      0.89         5
    security       0.80      0.80      0.80         5
social_policy      1.00      0.80      0.89         5
 environment       0.71      1.00      0.83         5

    accuracy                           0.85        20
   macro avg       0.88      0.85      0.85        20
weighted avg       0.88      0.85      0.85        20
```

Does the model matter? GPT-3.5 vs GPT-4o-mini vs GPT-5-mini

Early ChatGPT (GPT-3.5) and recent models can produce very different results on the same task. This can be also different with non-GPT models as Gemini and LlAMA.

```python
## run the same 20 test texts through GPT-3.5-turbo and GPT-5-mini
## we already have GPT-4o-mini results in llm_labels

import time

## same prompt for all three models
models_to_compare = {
    "gpt-3.5-turbo": [],
    "gpt-5-mini": [],
}

for model_name, label_list in models_to_compare.items():
    print(f"Running {model_name}...")
    for item in test:
        ## ask_gpt already accepts a model parameter
        label = ask_gpt(zero_shot_prompt, item["text"], model=model_name).strip().lower()
        label_list.append(label)
        time.sleep(0.3)
    print(f"  {model_name} done")

gpt35_labels = models_to_compare["gpt-3.5-turbo"]
gpt5_labels  = models_to_compare["gpt-5-mini"]
print("\nAll models done.")
```

```
Running gpt-3.5-turbo...
  gpt-3.5-turbo done
Running gpt-5-mini...
  gpt-5-mini done

All models done.
```

```python
## compare F1 across all three LLM models
from sklearn.metrics import f1_score, classification_report

human_labels = [d["human_label"] for d in test]
categories = ["economy", "security", "social_policy", "environment"]

## build a comparison table: model → macro F1 + per-category F1
model_results = {
    "GPT-3.5-turbo": gpt35_labels,
    "GPT-4o-mini":   llm_labels,
    "GPT-5-mini":    gpt5_labels,
}

print(f"{'Model':<22s} {'Macro F1':>9s}   {'economy':>8s} {'security':>9s} {'social':>8s} {'environ':>8s}")
print("-" * 75)

for name, preds in model_results.items():
    macro = f1_score(human_labels, preds, labels=categories, average="macro")
    per_cat = f1_score(human_labels, preds, labels=categories, average=None)
    print(f"  {name:<20s} {macro:>8.2f}   {per_cat[0]:>8.2f} {per_cat[1]:>8.2f} {per_cat[2]:>8.2f} {per_cat[3]:>8.2f}")
```

```
Model                  Macro F1    economy  security   social  environ
---------------------------------------------------------------------------
  GPT-3.5-turbo            0.85       0.80     0.80     0.89     0.91
  GPT-4o-mini              0.85       0.89     0.80     0.89     0.83
  GPT-5-mini               0.85       0.75     0.91     0.89     0.83
```

Inter-coder reliability is important

- You report how much your human coders agree with each other.
- You can also report human agreement vs model

For example **Cohen's Kappa** and **Krippendorf's Alpha** measures agreement corrected for chance.

> ⚠ **Reality check**
>
> This was 20 test texts. For a real study you'd need hundreds. LLMs cost money per call and their outputs can change between model versions. Benchmark multiple models. Consider open-source LLMs.

Scaling up: the Batch API

When you have thousands (or millions) of texts to classify, sending them one at a time through the API is slow and expensive.

For example, OpenAI's **Batch API** lets you submit many requests at once and get results later:

- **50% cheaper** than real-time API calls
- Results come back within 24 hours

Part 3: Named Entity Recognition — spaCy vs LLM

Now for some other things LLMs can do. NER finds people, places, and organizations in text. spaCy does this fast and free. However, it labels “Sweden” and “Geneva” both as `GPE` (geopolitical entity). It can’t tell you that Sweden is where the person is *from* and Geneva is where the event *happened*.

Why? spaCy uses statistical patterns. It labels entity *types* but can’t reason about context. LLMs read the whole sentence and understand *roles*.

```
For example:
sentence: "Senator Smith criticized President Macron for France's energy policy"

entities:
  Smith:  { type: PERSON, role: SPEAKER/CRITIC }
  Macron: { type: PERSON, role: TARGET }
  France: { type: GPE,    role: AFFILIATED_COUNTRY }
```

```python
## spaCy first — the standard NER tool
import spacy
from spacy import displacy

## load spaCy's medium model for NER
nlp_md = spacy.load("en_core_web_md")

## 4 texts chosen to show where spaCy struggles and LLMs shine
ner_texts = [
    "The Swedish foreign minister traveled to Geneva to discuss the crisis unfolding in Sudan.",
    "The suspect was described as speaking broken German with a Middle Eastern accent.",
    "The former Nigerian finance minister, now based in Washington, published a report on poverty in East Africa.",
    "French and German negotiators held talks in Vienna on sanctions targeting businesses in Tehran.",
]

## run NER on the first sentence and render the entities inline
doc = nlp_md(ner_texts[0])
displacy.render(doc, style="ent", jupyter=True)  ## color-coded entity labels
```

The Swedish NORP foreign minister traveled to Geneva GPE to discuss the crisis unfolding in Sudan GPE .

```python
## all entities across all texts
print("spaCy entities:\n")
spacy_results = []
for i, text in enumerate(ner_texts):
    doc = nlp_md(text)
    ## ent.text = the word, ent.label_ = the entity type (GPE, PERSON, etc.)
    ents = [(ent.text, ent.label_) for ent in doc.ents]
    spacy_results.append(ents)
    print(f"  Text {i+1}: {ents}")

print("Notice in Text 1: 'Swedish' (origin) and 'Geneva' (destination) both get the same label.")
```

```
spaCy entities:

  Text 1: [('Swedish', 'NORP'), ('Geneva', 'GPE'), ('Sudan', 'GPE')]
  Text 2: [('German', 'NORP'), ('Middle Eastern', 'LOC')]
  Text 3: [('Nigerian', 'NORP'), ('Washington', 'GPE'), ('East Africa', 'GPE')]
  Text 4: [('French', 'NORP'), ('German', 'NORP'), ('Vienna', 'GPE'), ('Tehran', 'GPE')]
Notice in Text 1: 'Swedish' (origin) and 'Geneva' (destination) both get the same label.
```

```python
## now ask GPT:

## the prompt asks for standard entity types PLUS a role taxonomy
## this is the part spaCy can't do, distinguishing *why* a place is mentioned
ner_prompt = """Extract all named entities from this text. For each entity return:
- text: the entity as it appears
- type: PERSON, GPE, NORP, or ORGANIZATION
- role: the entity's contextual role — one of:
  person_origin, event_location, person_base, affected_region, organization_base, subject_location, other

Return a JSON array."""

llm_entities = []
for text in ner_texts:
    raw = ask_gpt(ner_prompt, text, model="gpt-3.5-turbo")
    parsed = parse_json_response(raw)  ## strip code fences, parse as JSON
    llm_entities.append(parsed)
    time.sleep(0.3)

print("Text 1 — LLM entities WITH roles:\n")
for ent in llm_entities[0]:
    print(f"  {ent['text']:20s}  {ent['type']:12s}  role: {ent['role']}")
```

```
Text 1 — LLM entities WITH roles:

  Swedish              NORP          role: other
  Geneva               GPE           role: event_location
  Sudan                GPE           role: affected_region
```

Checking against ground truth

▶ Show NER ground truth

	text_#	entity	true_role	spaCy_found	LLM_found	LLM_role	role_correct
0	1	Swedish	person_origin	✓	✓	other	✗
1	1	Geneva	event_location	✓	✓	event_location	✓
2	1	Sudan	affected_region	✓	✓	affected_region	✓
3	2	German	subject_language	✓	✓	affected_region	✗
4	2	Middle Eastern	person_origin	✓	✓	affected_region	✗
5	3	Nigerian	person_origin	✓	✓	person_origin	✓
6	3	Washington	person_base	✓	✓	person_base	✓
7	3	East Africa	affected_region	✓	✓	affected_region	✓
8	4	French	person_origin	✓	✓	other	✗
9	4	German	person_origin	✓	✓	other	✗
10	4	Vienna	event_location	✓	✓	event_location	✓
11	4	Tehran	subject_location	✓	✓	affected_region	✗

```python
## quick summary
## recall = of all ground-truth entities, what fraction did each tool find?
spacy_recall = sum(1 for r in rows if r["spaCy_found"] == "✓") / len(rows)
llm_recall   = sum(1 for r in rows if r["LLM_found"] == "✓") / len(rows)
## role accuracy = of the entities the LLM found, how many roles did it get right?
role_assessed = [r for r in rows if r["role_correct"] in ("✓", "✗")]
role_accuracy = sum(1 for r in role_assessed if r["role_correct"] == "✓") / len(role_assessed) if role_assessed else 0

print(f"Entity detection recall:")
print(f"  spaCy: {spacy_recall:.0%}")
print(f"  LLM:   {llm_recall:.0%}")
print(f"\nRole accuracy (LLM only — spaCy has no roles):")
print(f"  LLM:   {role_accuracy:.0%} ({sum(1 for r in role_assessed if r['role_correct'] == '✓')}/{len(role_assessed)} correct)")
```

```
Entity detection recall:
  spaCy: 100%
  LLM:   100%

Role accuracy (LLM only — spaCy has no roles):
  LLM:   50% (6/12 correct)
```

Sumary

spaCy: fast, free, works offline. Good when you just need to *find* entities. But it can't tell you *why* a location is mentioned.

LLM: assigns roles. But it's slower, costs money, and can hallucinate entities that aren't there. Validation is key!

Part 4: Vision OCR — LLMs can read images

Another thing LLMs can do: read images. Archives are full of scanned documents: old typefaces, degraded paper. Vision LLMs can look at an image, read it, and output text.

Convential OCR models can struggle with non Latin text.

Let’s try GPT-4o and Gemini on a real historical document. Here we also need to validate.

There are different ways of doing so.

```python
import urllib.request

## download Executive Order 9981 — Truman's desegregation of the armed forces (1948)
## typewritten document, NARA public domain
img_url = "https://upload.wikimedia.org/wikipedia/commons/4/46/Executive_Order_9981.jpg"
req = urllib.request.Request(img_url, headers={"User-Agent": "LLM-Workshop/1.0 (educational use)"})
with urllib.request.urlopen(req) as response, open("executive_order_9981.jpg", "wb") as f:
    f.write(response.read())
print("Downloaded: executive_order_9981.jpg")
```

```
Downloaded: executive_order_9981.jpg
```

▶ Show Executive Order ground truth

```
Ground truth: 431 words
```

```python
## show the image
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

img = mpimg.imread("executive_order_9981.jpg")
plt.figure(figsize=(10, 8))
plt.imshow(img)
plt.axis("off")
plt.title("Executive Order 9981 — Desegregation of the Armed Forces (1948)")
plt.show()
```

Executive Order 9981 — Desegregation of the Armed Forces (1948)

EXECUTIVE ORDER

ESTABLISHING THE PRESIDENT'S COMMITTEE ON EQUALITY OF TREATMENT AND OPPORTUNITY IN THE ARMED SERVICES

WHEREAS it is essential that there be maintained in the armed services of the United States the highest standards of democracy, with equality of treatment and opportunity for all those who serve in our country's defense:

NOW, THEREFORE, by virtue of the authority vested in me as President of the United States, by the Constitution and the statutes of the United States, and as Commander in Chief of the armed services, it is hereby ordered as follows:

1. It is hereby declared to be the policy of the President that there shall be equality of treatment and opportunity for all persons in the armed services without regard to race, color, religion or national origin. This policy shall be put into effect as rapidly as possible, having due regard to the time required to effectuate any necessary changes without impairing efficiency or morale.

2. There shall be created in the National Military Establishment an advisory committee to be known as the President's Committee on Equality of Treatment and Opportunity in the Armed Services, which shall be composed of seven members to be designated by the President.

3. The Committee is authorized on behalf of the President to examine into the rules, procedures and practices of the armed services in order to determine in what respect such rules, procedures and practices may be altered or improved with a view to carrying out the policy of this order. The Committee shall confer and advise with the Secretary of Defense, the Secretary

of the Army, the Secretary of the Navy, and the Secretary of the Air Force, and shall make such recommendations to the President and to said Secretaries as in the judgment of the Committee will effectuate the policy hereof.

4. All executive departments and agencies of the Federal Government are authorized and directed to cooperate with the Committee in its work, and to furnish the Committee such information or the services of such persons as the Committee may require in the performance of its duties.

5. When requested by the Committee to do so, persons in the armed services or in any of the executive departments and agencies of the Federal Government shall testify before the Committee and shall make available for the use of the Committee such documents and other information as the Committee may require.

6. The Committee shall continue to exist until such time as the President shall terminate its existence by Executive order.

Harry Truman

THE WHITE HOUSE,
July 26, 1948.

THE NATIONAL ARCHIVES
FILED AND MADE AVAILABLE FOR PUBLIC INSPECTION
JUL 27 10 39 AM '48
IN THE DIVISION OF THE FEDERAL REGISTER

9981

```python
import base64

## the API needs the image as a base64 string (text encoding of binary data)
with open("executive_order_9981.jpg", "rb") as f:
    img_b64 = base64.b64encode(f.read()).decode()

ocr_prompt = ("Extract all the text from this historical document image. "
              "Return only the extracted text, preserving the original structure.")

## GPT-4o vision — send text prompt + image together in one message
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{
        "role": "user",
        ## content is a list: one text block + one image block
        "content": [
            {"type": "text", "text": ocr_prompt},
            {"type": "image_url", "image_url": {"url": f"{img_b64}"}}
        ]
    }]
)
gpt_ocr = response.choices[0].message.content

print("GPT-4o extracted text (first 500 chars):\n")
print(gpt_ocr[:500])
```

```
GPT-4o extracted text (first 500 chars):

EXECUTIVE ORDER

ESTABLISHING THE PRESIDENT'S COMMITTEE ON
EQUALITY OF TREATMENT AND OPPORTUNITY IN
THE ARMED SERVICES

WHEREAS it is essential that there be maintained in the
armed services of the United States the highest standards of
democracy, with equality of treatment and opportunity for all
those who serve in our country's defense:

NOW, THEREFORE, by virtue of the authority vested in
me as President of the United States, by the Constitution and the
statutes of the United States, and as C
```

```python
import PIL.Image

## Gemini vision — same image, same prompt
## Gemini takes a PIL Image object directly (no base64 needed)
img_pil = PIL.Image.open("executive_order_9981.jpg")
gemini_model = genai.GenerativeModel("gemini-2.0-flash")
response = gemini_model.generate_content([ocr_prompt, img_pil])
gemini_ocr = response.text

print("Gemini extracted text (first 500 chars):\n")
print(gemini_ocr[:500])
```

```
Gemini extracted text (first 500 chars):

EXECUTIVE ORDER
-2-
ESTABLISHING THE PRESIDENT'S COMMITTEE ON
EQUALITY OF TREATMENT AND OPPORTUNITY IN
THE ARMED SERVICES
WHEREAS it is essential that there be maintained in the
armed services of the United States the highest standards of
democracy, with equality of treatment and opportunity for all
those who serve in our country's defense:
NOW, THEREFORE, by virtue of the authority vested in
me as President of the United States, by the Constitution and the
statutes of the United States, and as
```

How did they do?

Let’s measure against the known transcription.

Validating LLM Transcription

We compare each model's output against a human ground truth transcript.

- **Precision** — are the extracted words real?
- **Recall** — were all real words recovered?
- **F1** — overall balance between the two

> We ask: *did the right words appear?*

This is a very simple way of validating this output: there are many more.

```python
def word_level_metrics(extracted, ground_truth):
    # --- preprocessing: lowercase + tokenize into unique word sets ---
    # NOTE: using sets loses word frequency — "the the the" counts same as "the"
    truth_words = set(ground_truth.lower().split())
    extracted_words = set(extracted.lower().split())

    # guard: if ground truth is empty, metrics are undefined
    if not truth_words:
        return {"precision": 0, "recall": 0, "f1": 0}

    # true positives: words present in both outputs
    overlap = truth_words & extracted_words

    # precision: of all words the LLM returned, how many were real?
    # low precision → model is hallucinating words not in the original
    precision = len(overlap) / len(extracted_words) if extracted_words else 0

    # recall: of all real words, how many did the LLM recover?
    # low recall → model is dropping words (under-extraction)
    recall = len(overlap) / len(truth_words)

    # harmonic mean of precision and recall
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0

    return {
        "precision":          round(precision, 3),
        "recall":             round(recall, 3),
        "f1":                 round(f1, 3),
        "words_in_truth":     len(truth_words),
        "words_extracted":    len(extracted_words),
        "words_correct":      len(overlap),
        "words_missed":       len(missed := truth_words - extracted_words),   # in truth, not extracted
        "words_hallucinated": len(extracted_words - truth_words)              # extracted, not in truth
    }

# --- run for each model ---
results_ocr = {
    "GPT-4o": word_level_metrics(gpt_ocr, ground_truth),
    "Gemini": word_level_metrics(gemini_ocr, ground_truth)
}

# --- reshape: models as rows, metrics as columns ---
df_ocr = pd.DataFrame(results_ocr).T

# --- heatmap coloring on core metrics only: red=bad, green=good ---
df_ocr.style.background_gradient(
```

	precision	recall	f1	words_in_truth	words_extracted	words_correct	words_missed	words_hallucinated
GPT-4o	0.870000	0.955000	0.911000	176.000000	193.000000	168.000000	8.000000	25.000000
Gemini	0.857000	0.955000	0.903000	176.000000	196.000000	168.000000	8.000000	28.000000

```python
## what words did each model miss?
truth_words = set(ground_truth.lower().split())
gpt_words = set(gpt_ocr.lower().split())
gem_words = set(gemini_ocr.lower().split())

## words that BOTH models missed — likely the hardest parts of the document
both_missed = truth_words - gpt_words - gem_words
if both_missed:
    print(f"Words both models missed ({len(both_missed)}): {sorted(both_missed)[:20]}")

## words where the models disagreed
gpt_only = (gpt_words & truth_words) - gem_words
gem_only = (gem_words & truth_words) - gpt_words
if gpt_only:
    print(f"\nOnly GPT-4o got these right ({len(gpt_only)}): {sorted(gpt_only)[:10]}")
if gem_only:
    print(f"Only Gemini got these right ({len(gem_only)}): {sorted(gem_only)[:10]}")
```

```
Words both models missed (8): ['1948', 'cooperate', 'designated', 'establishment', 'harry', 's.', 'services.', 'truman']
```

There are other ways to validate

Thank you so much!

It was a pleasure visiting Gothenburg :)